

SmartGym Adaptive Training System

# Guia de Medicion y Calibracion

ROM, calibracion, errores de senal, validacion y debugging.

## Version 1.0 Prototype

ESP32-S3 firmware | Firebase RTDB | React dashboard

Generated documentation package

SmartGym Adaptive Training System

# Guia de Medicion y Calibracion

ROM, calibracion, errores de senal, validacion y debugging.

## Version 1.0 Prototype

ESP32-S3 firmware | Firebase RTDB | React dashboard

Generated documentation package

# Guia De Medicion Y Calibracion SmartGym

Idioma: [English](MEASUREMENT\_AND\_CALIBRATION\_GUIDE.md) | Espanol

Esta guia explica como SmartGym mide movimiento, convierte lecturas de sensor en valores utiles, calibra ROM, maneja carga de pin y detecta errores comunes. Esta escrita para alguien que no construyo el proyecto y necesita entenderlo de extremo a extremo.

## Proposito

Durante un entrenamiento, el firmware debe responder:

- Donde esta la maquina en su recorrido.
- Si el usuario esta subiendo o bajando.
- Si acaba de ocurrir una repeticion valida.
- Que carga fisica tenia la maquina.
- Que recomendacion debe mostrarse despues.

Para eso, SmartGym combina:

- Sensor analogico de movimiento.
- Calibracion ROM especifica por usuario.
- Maquina de estados para repeticiones.
- Modelo de carga de pin de maquina.
- Registro local de sesion.
- Sincronizacion con Firebase.

## Vista General

El pipeline de medicion es:

1. Muestrear el sensor analogico cada 5 ms.
2. Suavizar la lectura con filtro EMA.
3. Convertir la lectura filtrada en posicion de maquina.
4. Normalizar la posicion a porcentaje ROM usando bottom/top calibrados.
5. Calcular velocidad desde el cambio de posicion normalizada.
6. Ejecutar la maquina de estados de repeticion.
7. Registrar reps, series, tiempos, pin load y metricas.
8. Guardar la sesion localmente y subirla a Firebase por fases.

## Entradas De Sensor

## Sensor De Movimiento

El firmware principal usa un sensor analogico conectado a GPIO 17 en el ESP32-S3. La lectura se realiza cada 5 ms:

```
sample_interval = 0.005 s  
sample_rate = 1 / 0.005 = 200 Hz
```

Esta frecuencia permite resolver movimiento normal de gimnasio mientras el ESP32-S3 tambien ejecuta LVGL y Firebase.

## RFID

RFID identifica al usuario activo. El firmware necesita el usuario para cargar calibracion ROM y recomendacion correctas.

## Pin Load

El firmware trata el pin load como estado de la maquina, no del usuario. En el firmware principal, el usuario actualiza el valor con botones de pantalla. La carpeta 'weight\_detection/' contiene un prototipo VL53L0X separado para deteccion automatica del stack.

## De Senal Cruda A Posicion

El ADC del ESP32-S3 es de 12 bits, con rango:

```
0 a 4095 counts
```

Si el recorrido completo se modela como 2000 mm, la resolucion teorica es:

```
resolution_mm = 2000 / 4096 = 0.488 mm por count
```

Un modelo conceptual de conversion es:

```
position_mm = (adc_or_voltage / full_scale) * stroke_length_mm
```

Donde:

- 'position\_mm' es la posicion estimada.
- 'adc\_or\_voltage' es la senal medida.
- 'full\_scale' es la referencia de escala completa.
- 'stroke\_length\_mm' es el recorrido asumido.

## Filtrado

La senal se suaviza con un promedio movil exponencial:

```
filtered[k] = alpha * raw[k] + (1 - alpha) * filtered[k - 1]
```

Con un valor alrededor de:

```
alpha = 0.50
```

El filtro reduce ruido ADC, vibracion mecanica y cambios falsos de pendiente. Demasiado poco filtrado genera reps falsas; demasiado filtrado introduce retraso.

## Calibracion ROM

La calibracion es especifica por usuario y tipo de maquina. Guarda un punto inferior y superior.

Flujo:

1. Mover a posicion inferior y confirmar.
2. Mover a posicion superior y confirmar.
3. Confirmar pin load actual.
4. Hacer repeticiones controladas.
5. Guardar resultado y recomendacion.

## Normalizacion ROM

Con bottom y top, el firmware calcula:

```
rom_percent = ((x - x_bottom) / (x_top - x_bottom)) * 100
```

Donde:

- 'x' es la posicion actual filtrada.
- 'x\_bottom' es el punto inferior calibrado.
- 'x\_top' es el punto superior calibrado.

Esta es la idea central del sistema: convertir desplazamiento crudo en un porcentaje relativo al usuario y a la maquina.

## Por Que La Calibracion Es Por Usuario

Los usuarios no siempre usan los mismos extremos mecanicos:

- diferente longitud corporal
- diferente setup
- posicion de asiento o agarre distinta
- preferencia de no bloquear completamente la articulacion

Sin calibracion, un buen movimiento puede verse como ROM insuficiente o una repeticion mala puede parecer valida.

## Calculo De Velocidad

La velocidad sale del cambio entre muestras:

$$\text{velocity} = (x[k] - x[k - 1]) / dt$$

Si se usa ROM normalizado:

$$\text{velocity\_rom} = (\text{rom}[k] - \text{rom}[k - 1]) / dt$$

Con 'dt' cercano a '0.005 s'. La velocidad ayuda a detectar fases y calidad de movimiento.

## Deteccion De Repeticiones

El firmware usa una maquina de estados basada en ROM:

- 'Bottom'
- 'Ascending'
- 'Top'
- 'Descending'

Logica simplificada:

1. En bottom espera movimiento real hacia arriba.
2. Si ROM sube consistentemente, entra a ascending.
3. Si alcanza region superior, entra a top.
4. Si ROM baja, entra a descending.
5. Si vuelve abajo y pasa validaciones, cuenta la repeticion.

Una repeticion valida requiere:

- ROM suficiente.
- Llegar cerca del top calibrado.
- Duracion razonable.

- Orden de fases coherente.
- Senal no excesivamente ruidosa.

## Por Que Se Rechazan Reps

Una repeticion puede rechazarse si:

- no alcanza suficiente ROM
- no llega arriba
- se revierte demasiado temprano
- hay ruido fuerte
- dura demasiado poco

Por eso mover la maquina no siempre significa contar una repeticion.

## Pin Load Vs Recomendacion

Pin load es la posicion fisica del selector de peso. Pertenece a la maquina.

Recomendacion es consejo especifico por usuario, calculado desde calibracion o historial.

Ejemplo:

```
pin load = 20 kg  
recomendacion = 25 kg
```

La sesion inicia con 20 kg si el usuario no mueve el pin. El software no puede asumir que el hardware cambio.

## Recomendacion De Carga

El modelo conceptual es:

$$v = a + bL$$

Donde:

- 'v' es velocidad representativa.
- 'L' es carga.
- 'a' y 'b' son constantes ajustadas.

Con datos de velocidad a cargas conocidas, el sistema puede sugerir una siguiente carga.

## Resultado De Calibracion

Una calibracion debe guardar:

- bottom calibrado
- top calibrado
- confianza/calidad
- carga probada
- recomendacion siguiente
- machine type
- user id

Ruta Firebase:

```
calibrations/{uid}/{machineTypeId}
```

## Flujo Firebase

Al terminar una sesion:

1. El firmware crea el resultado local.
2. Lo guarda en una cola NVS.
3. Un servicio de sync sube por fases.

Fases tipicas:

- session root
- metadata diaria
- timeline
- day summary
- week summary
- set details
- rep sets

Esto evita perder datos y reduce riesgo de bloqueos por payloads grandes.

## Restricciones De Memoria

El ESP32-S3 debe balancear:

- UI LVGL rica
- escrituras Firebase/TLS

Por eso:



- se liberan pantallas opcionales antes de subidas pesadas
- se fragmentan payloads cuando hay poco heap
- se usa cola NVS para reintento
- repSets se dividen en escrituras mas pequenas

## Fuentes De Error

### Error De Alineacion Mecanica

Un sensor o soporte mal alineado causa:

- rango comprimido
- top/bottom inconsistentes
- lecturas ToF malas

### Ruido ADC

Puede causar:

- grafica inestable
- transiciones falsas
- velocidad inconsistente

### Mala Calibracion

Si bottom/top se guardan mal:

- ROM se vuelve enganoso
- reps buenas pueden rechazarse
- top o bottom nunca se alcanzan en software

### Tecnica Del Usuario

Parciales, rebotes, tempo irregular o setup distinto cambian la senal.

### Falla De Red

La sesion puede terminar bien aunque la subida se retrase. Para eso existe la cola local.

### Presion De Memoria

El dispositivo puede tener CPU disponible pero no heap seguro para Firebase. Aparecen delays, chunking o backoff.

## Validar Una Instalacion Nueva

### Hardware

1. Confirma que el sensor de movimiento esta firme.

2. Confirma pantalla y touch.
3. Confirma RFID.
4. Confirma montaje ToF si se usa.

## Senal

1. Mira la grafica al mover la maquina lentamente.
2. Confirma direccion correcta.
3. Confirma que no satura antes de tiempo.
4. Confirma que llega a zonas baja y alta.

## Calibracion

1. Guarda bottom.
2. Guarda top.
3. Mueve la maquina otra vez.
4. Confirma ROM cerca de 0% a 100%.
5. Haz reps suaves y confirma que cuentan.

## Sesion

1. Escanea RFID.
2. Espera user sync.
3. Inicia sesion.
4. Haz reps.
5. Termina sesion.
6. Confirma resumen.
7. Confirma subida Firebase.

## Depurar ROM Incorrecto

1. Verifica si bottom/top se guardaron al revés.
2. Revisa que la maquina llegue a extremos esperados.
3. Revisa si el sensor esta flojo.
4. Observa ruido en grafica.
5. Recalibra con movimiento lento y completo.

## Depurar Pin Load Incorrecto

1. Verifica si el firmware principal esta en modo manual.
2. Confirma que el usuario actualizo botones en pantalla.
3. Si usas ToF, recalibra rangos de distancia.
4. Revisa alineacion del target y holder.

## Explicacion Corta Para Un Nuevo Integrante

SmartGym muestrea movimiento cada 5 ms, suaviza la senal, la convierte a ROM normalizado, detecta fases de repeticion, registra reps y sets con el pin load actual, y sube la sesion terminada a Firebase mediante una cola segura.

Si alguien entiende esa frase, ya tiene el modelo mental para navegar el sistema.

## Documentos Relacionados

- [README del proyecto](../README.es.md)
- [Arquitectura del sistema](SYSTEM\_ARCHITECTURE.es.md)
- [Guia de desarrollador](DEVELOPER\_GUIDE.es.md)
- [Guia de Firebase](FIREBASE\_GUIDE.es.md)
- [Troubleshooting](TROUBLESHOOTING.es.md)
- [Prototipo de deteccion de peso](../weight\_detection/README.es.md)